Lab 10
Lighting in OpenGL

Objectives

- We will learn to implement lighting in OpenGL.

Programming Exercises (90 minutes)

In this lab, we will learn how to do lighting in OpenGL, an overview of the tasks in this lab is as follows:

Task 1: Setting up lighting in OpenGL
- *Task 1A: Setting up the world* – define and setup the world and the geometry of the lights
- *Task 1B: Setting up lighting mode* - enable and initialize lighting
- *Task 1C: Setting up a light* – Set up a directional light

Task 2: Creating More Lights
- *Task 2A: Creating spot lights*
- *Task 2B: Creating point lights*

Task 3: Turning on and off lights

Task 4: Practice – Flying Light (Firefly)!

In all previous labs, you have been using the `CGLabs.cbp`, `CGLabmain.cpp` and `CGLabmain.hpp` as the base for your program. In this lab (Lab10) and also Lab11 later, we will construct a new project and call it `CGLab10and11.cbp`, which consists of 6 files in the project as follows:

- `CGLabmain.hpp` (same as before, no modification needed)
- `CGLab10and11main.cpp` (modified from `CGLabmain.cpp`, see Task 1A)
- `CGLab10and11.hpp` (to be created in Task 1A)
- `CGLab10and11.cpp` (to be created in Task 1A)
- `CGLab09.hpp` (given below)
- `CGLab09.cpp` (given below)

Please refer back to Lab01 Instruction on how to create a new project.

To make our virtual world more interesting, we will populate our virtual world with the `MyFan` and `MyMovingSmiley` objects created in Lab 09. If you have completed the exercises in Lab 09, you may use your own version, if not the following code (as given in the Lab09 instructions) is given to you below.

CGLab09.hpp:

```
/*
 TCG6223 Computer Graphics
 CGLab09.hpp

 Objective: Header File for Lab09 on Animation

 Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>

 INSTRUCTIONS
 ============
 Please refer to CGLabmain.cpp for instructions
*/

#ifndef YP_CGLAB09_HPP
#define YP_CGLAB09_HPP
#include <GL/glut.h>
#include "CGLabmain.hpp"

namespace CGLab09 {

class MyMovingSmiley
{
 public:
    MyMovingSmiley();
    ~MyMovingSmiley();

    void draw();
    void tickTime(long int elapseTime);

 private:

    GLUquadricObj *pObj;
    GLfloat posx, posy, posz;
    GLfloat roty;

    GLfloat velx, vely, velz; //unit per sec
};

class MyFan
{
 public:
    MyFan();
    ~MyFan();

    void tickTime(long int elapseTime);
    void draw();

 private:
    void drawBase();
    void drawMiddle();
    void drawWings();

    GLUquadricObj *pObj;
    GLfloat pitchangle;
    GLfloat swingangle, wingsangle; //in degree
    GLfloat swingspeed;  //in degree per sec
    GLfloat wingsspeed;  //in degree per sec
};
```

```
//-------------------------------------
//the main program will call methods from this class
class MyVirtualWorld
{
 public:
    MyMovingSmiley mymovingsmiley;
    MyFan myfan;
    long int timeold, timenew, elapseTime;

    void draw()
    {
     drawFloor();
     mymovingsmiley.draw();
     myfan.draw();
    }

    void drawFloor()
    {
     glDisable(GL_CULL_FACE);

     glColor3f(0.0f, 0.0f, 1.0f);
     glBegin(GL_QUADS);
        glVertex3f( 19.0f, 0.0f, 19.0f);
        glVertex3f( 19.0f, 0.0f,-19.0f);
        glVertex3f(-19.0f, 0.0f,-19.0f);
        glVertex3f(-19.0f, 0.0f, 19.0f);
     glEnd();
     glEnable(GL_CULL_FACE);
    }

    //NOTE: different compare with those in earlier lab
    void tickTime()
    {
     timenew    = glutGet(GLUT_ELAPSED_TIME);
     elapseTime = timenew - timeold;
     timeold    = timenew;

     mymovingsmiley.tickTime(elapseTime);
     myfan.tickTime(elapseTime);
    }

    //for any one-time only initialization of the
    //   virtual world before any rendering takes place
    //   BUT after OpenGL has been initialized
    void init()
    {
     timeold = glutGet(GLUT_ELAPSED_TIME);
    }
};

}; //end of namespace CGLab09

#endif //YP_CGLAB09_HPP
```

CGLab09.cpp:

```
/*
 TCG6223 Computer Graphics

 CGLab09.cpp

 Objective: Lab09 on Animation

 Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>

 INSTRUCTIONS
 ============
 Please refer to CGLabmain.cpp for instructions
*/

#include <GL/glut.h>
#include "CGLab09.hpp"

using namespace CGLab09;

MyMovingSmiley::MyMovingSmiley()
{
 //Setup Quadric Object
 pObj = gluNewQuadric();
 gluQuadricNormals(pObj, GLU_SMOOTH);

 posx = -5.0f; posy = 0.0f; posz = 2.0f;
 roty = 30.0f;

 //initial velocity (in unit per second)
 velx=40.0f;
 vely= 0.0f;
 velz=30.0f;
}

MyMovingSmiley::~MyMovingSmiley()
{
 gluDeleteQuadric(pObj);
}

void MyMovingSmiley::draw()
{
 glPushMatrix();
    glTranslatef(posx, posy, posz);
    glRotatef(roty, 0.0f, 1.0f, 0.0f);

    glPushMatrix();
       //head
       glTranslatef(0.0f, 4.0f, 0.0f);
       glColor3f(1.0f, 1.0f, 0.0f);
       gluSphere(pObj,4.0f, 20, 20);
       //nose
       glTranslatef(0.0f, 0.0f, 4.0f);
       glColor3f(1.0f, 0.5f, 0.0f);
       gluCylinder(pObj,1.0f, 0.0f, 2.0f, 5, 5);
       //eyes
       glPushMatrix();
          glTranslatef(1.5f, 1.5f, -1.2f);
          glColor3f(1.0f, 1.0f, 1.0f);
```

```
          gluSphere(pObj,1.0f, 10, 10);
          glTranslatef(0.0f, 0.2f, 0.5f);
          glColor3f(0.0f, 0.5f, 1.0f);
          gluSphere(pObj,0.7f, 10, 10);
       glPopMatrix();
       glPushMatrix();
          glTranslatef(-1.5f, 1.5f, -1.2f);
          glColor3f(1.0f, 1.0f, 1.0f);
          gluSphere(pObj,1.0f, 10, 10);
          glTranslatef(0.0f, 0.2f, 0.5f);
          glColor3f(0.0f, 0.5f, 1.0f);
          gluSphere(pObj,0.7f, 10, 10);
       glPopMatrix();
       //mouth
       GLboolean cullingIsOn;
       glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
       glDisable(GL_CULL_FACE);
       glTranslatef(0.0f, -1.8f, -1.2f);
       glRotatef(-45, 1.0f, 0.0f, 0.0f);
       glColor3f(1.0f, 0.0f, 0.0f);
       gluDisk(pObj,0.0f, 1.5f, 10, 10);
       if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
    glPopMatrix();
 glPopMatrix();
}

void MyMovingSmiley::tickTime(long int elapseTime) //elapsetime in milisec
{
 float elapseTimeInSec = elapseTime / 1000.0f;
 posx += elapseTimeInSec * velx;
 posy += elapseTimeInSec * vely;
 posz += elapseTimeInSec * velz;

 if (posx > 15)
 {
    posx = 30 - posx;  //posx = 15 - (posx - 15)
    velx = -velx;
 }
 else if (posx<-15)
 {
    posx = -30 - posx; //posx = -15 - (posx -(-15))
    velx = -velx;
 }

 if (posz > 15)
 {
    posz = 30 - posz;
    velz = -velz;
 }
 else if (posz<-15)
 {
    posz = -30 - posz;
    velz = -velz;
 }

};

MyFan::MyFan()
{
 //Setup Quadric Object
```

```
 pObj = gluNewQuadric();
 gluQuadricNormals(pObj, GLU_SMOOTH);
 pitchangle=-20.0f;
 swingangle= 45.0f;
 wingsangle= 30.0f;
 swingspeed = 90.0;  //degree per sec
 wingsspeed = 720.0;  //degree per sec
}

MyFan::~MyFan()
{
 gluDeleteQuadric(pObj);
}

void MyFan::draw()
{
 glPushMatrix();
    drawBase();
    glTranslatef(0.0f, 10.0f, 0.0f);
    glRotatef(-45, 0.0f, 1.0f, 0.0f);
    glRotatef(swingangle, 0.0f, 1.0f, 0.0f);
    glRotatef(pitchangle, 1.0f, 0.0f, 0.0f);
    glTranslatef(0.0f,  0.0f, 2.0f);
    drawMiddle();
    glTranslatef(0.0f, 0.0f, 6.0f);
    glRotatef(wingsangle, 0.0f, 0.0f, 1.0f);
    drawWings();
 glPopMatrix();
}

void MyFan::drawBase()
{
 glPushMatrix();
    glRotatef(-90.0f, 1.0f, 0.0f, 0.0f);
    glColor3f(1.0f, 0.0f, 0.0f);
    gluCylinder(pObj, 5.0f, 1.0f, 2.0f, 10, 10);
    glColor3f(1.0f, 0.0f, 1.0f);
    gluCylinder(pObj, 1.0f, 1.0f, 10.0f, 10, 10);
    glRotatef(180.0f, 1.0f, 0.0f, 0.0f);
    gluDisk(pObj, 0.0f, 5.0f, 10, 10);
 glPopMatrix();
}

void MyFan::drawMiddle()
{
 glPushMatrix();
    glScalef(0.3f, 0.3f, 1.0f);
    glColor3f(1.0f, 1.0f, 0.0f);
    gluSphere(pObj, 5.0f, 10, 10);
    glTranslatef(0.0f, 0.0f, 5.0f);
    gluCylinder(pObj, 0.5f, 0.5f, 1.5f, 10, 10);
 glPopMatrix();
}

void MyFan::drawWings()
{
 GLboolean cullingIsOn;
 glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
 glDisable(GL_CULL_FACE);
```

```
 GLboolean normalizeIsOn;
 glGetBooleanv(GL_NORMALIZE, &normalizeIsOn);
 glEnable(GL_NORMALIZE);

 glColor3f(0.5f, 1.0f, 0.5f);
 glPushMatrix();
    glBegin(GL_TRIANGLES);
       glNormal3f( 1.0f, 0.0f, 2.0f);
       glVertex3f( 0.0f, 0.0f, 0.0f);
       glVertex3f( 0.0f, 4.0f, 0.0f);
       glVertex3f(-2.0f, 4.0f, 1.0f);

       glNormal3f( 0.0f, 1.0f, 2.0f);
       glVertex3f( 0.0f, 0.0f, 0.0f);
       glVertex3f(-4.0f, 0.0f, 0.0f);
       glVertex3f(-4.0f,-2.0f, 1.0f);

       glNormal3f(-1.0f, 0.0f, 2.0f);
       glVertex3f( 0.0f, 0.0f, 0.0f);
       glVertex3f( 0.0f,-4.0f, 0.0f);
       glVertex3f( 2.0f,-4.0f, 1.0f);

       glNormal3f( 0.0f,-1.0f, 2.0f);
       glVertex3f( 0.0f, 0.0f, 0.0f);
       glVertex3f( 4.0f, 0.0f, 0.0f);
       glVertex3f( 4.0f, 2.0f, 1.0f);
    glEnd();
 glPopMatrix();
 if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
 if (normalizeIsOn==GL_TRUE) glEnable(GL_NORMALIZE);
}

void MyFan::tickTime(long int elapseTime) //elapsetime in milisec
{
 //to be filled up by you
};
```

Task 1: Setting up lighting in OpenGL *(30 minutes)*

Task 1A: Setting up the world

We are now ready to write the new `CGLab10and11main.cpp` which will be modified from `CGLabmain.cpp`, instead of asking you to modify the program parts by parts, the whole `CGLab10and11main.cpp` program is show below with the modified part in bold. Please take note that most of the modification has nothing to do with setting up of lighting, but the modification allows us to be able to move the spotlights defined later. Please read the comments on what the modifications are about.

`CGLab10and11main.cpp`:

```
/*
 TCG6223 Computer Graphics

 CGLab10and11main.cpp

  Objective: Main Program for:
                Lab10 on Lighting &
                Lab11 on Textures

 Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>

 INSTRUCTIONS
 ============
 Please refer to CGLabmain.cpp for instructions

 SPECIAL NOTES
 =============
 * This main program is only for Lab10 and Lab11,
   For Lab01 to Lab09, use CGLabmain.cpp
 * This program is very similar to CGLabmain.cpp,
   differences are marked in between lines shown below:
       //BEGIN LAB10 Specific
       //END LAB10 Specific
   the same goes with Lab11 as well.
   Note that Lab11 built on Lab10 by adding texture features.

 CHANGE LOG
 ==========

 TO DO
 =====
*/

#include <cstdlib>
#include <iostream>
#include <iomanip>
#include <cmath>
#include <GL/glut.h>

#include "CGLabmain.hpp"

#include "CGLab10and11.hpp"

CGLab11and11::MyVirtualWorld myvirtualworld;

//BEGIN LAB10 Specific - define local axis for the spotlight and
```

```
//                        allow us to move the spot lights around
MyWorld  local;
MyAxis   spotlightsaxis;

enum MyMovementType
{
 movenone, moveworld, movelocal
};

MyMovementType movementType;
//END LAB10 Specific

using namespace std;

MyWindow   window;
MyWorld    world;
MyViewer   viewer;
MySetting  setting;
MyAxis     worldaxis;

void myDisplayFunc(void)
{
 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

 glPushMatrix();

    glTranslatef(world.posX, world.posY, world.posZ);
    glRotatef(world.rotateX, 1.0f, 0.0f, 0.0f);
    glRotatef(world.rotateY, 0.0f, 1.0f, 0.0f);
    glRotatef(world.rotateZ, 0.0f, 0.0f, 1.0f);
    glScalef(world.scaleX, world.scaleY, world.scaleZ);

    worldaxis.draw();

    myvirtualworld.draw();

    //BEGIN LAB10 Specific – define local axis for the spotlight and
    //                       allow us to move the spot lights around
    glPushMatrix();
       glTranslatef(0.0f, 20.0f, 0.0f);

       glTranslatef(local.posX, local.posY, local.posZ);
       glRotatef(local.rotateX, 1.0f, 0.0f, 0.0f);
       glRotatef(local.rotateY, 0.0f, 1.0f, 0.0f);
       glRotatef(local.rotateZ, 0.0f, 0.0f, 1.0f);
       glScalef(local.scaleX, local.scaleY, local.scaleZ);

       spotlightsaxis.draw();

       myvirtualworld.drawSpotLights();
    glPopMatrix();
    //END LAB10 Specific

 glPopMatrix();

 glFlush();   // send any buffered output to be rendered
 glutSwapBuffers();

 //Same as in Lab09
 myvirtualworld.tickTime();  //tick the clock
```

```
 glutPostRedisplay(); //force openGL to call myDisplayFunc() again
}

void myReshapeFunc(int width, int height)
{
 window.width  = width;
 window.height = height;
 glViewport(0, 0, width, height);
}

void myKeyboardFunc(unsigned char key, int x, int y)
{
 GLfloat xinc,yinc,zinc;
 xinc = yinc = zinc = 0.0;
 switch (key)
 {
    case 'a': case 'A': xinc = -setting.posInc;  break;
    case 'd': case 'D': xinc =  setting.posInc;  break;
    case 'q': case 'Q': yinc = -setting.posInc;  break;
    case 'e': case 'E': yinc =  setting.posInc;  break;
    case 'w': case 'W': zinc = -setting.posInc;  break;
    case 's': case 'S': zinc =  setting.posInc;  break;

    //BEGIN LAB10 Specific - define local axis for the spotlight and
    //                       allow us to move the spot lights around
    case '1': movementType = moveworld;
              break;
    case '2': movementType = movelocal;
              break;
    //END LAB10 Specific

    case 27 : exit(1); break;
 }

 //BEGIN LAB10 Specific - define local axis for the spotlight and
 //                       allow us to move the spot lights around
 //world.move(xinc, yinc, zinc);
 switch (movementType)
 {
    case moveworld: world.move(xinc, yinc, zinc);
                    break;
    case movelocal: local.move(xinc, yinc, zinc);
                    break;
 }
 //END LAB10 Specific

 glutPostRedisplay();
}

void mySpecialFunc(int key, int x, int y)
{
 switch (key)
 {
    case GLUT_KEY_DOWN  : world.rotateX -= setting.angleInc;  break;
    case GLUT_KEY_UP    : world.rotateX += setting.angleInc;  break;
    case GLUT_KEY_LEFT  : world.rotateY -= setting.angleInc;  break;
    case GLUT_KEY_RIGHT : world.rotateY += setting.angleInc;  break;
    case GLUT_KEY_HOME  : myDataInit(); break;
      case GLUT_KEY_F1    : setting.shadingMode = !setting.shadingMode;
                            if (setting.shadingMode)
```

```
                              glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
                            else
                                glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
                          break;
      case GLUT_KEY_F2    : worldaxis.toggle();
                          //BEGIN LAB10 Specific - toggle the axis of spotlight
                          spotlightsaxis.toggle();
                          //END LAB10 Specific
                            break;
      case GLUT_KEY_F3    : GLboolean lightingIsOn;
                          glGetBooleanv(GL_LIGHTING, &lightingIsOn);
                          if (lightingIsOn==GL_TRUE)
                             glDisable(GL_LIGHTING);
                             else  glEnable(GL_LIGHTING);
                          break;
 }

 glutPostRedisplay();
}

void myMouseFunc(int button, int state, int x, int y)
{
 y = window.height - y;
 switch (button)
 {
    case GLUT_RIGHT_BUTTON:
       if (state==GLUT_DOWN && !setting.mouseRightMode)
       {
          setting.mouseX = x;
          setting.mouseY = y;
          setting.mouseRightMode = true;
       }
       if (state==GLUT_UP && setting.mouseRightMode)
       {
          setting.mouseRightMode = false;
       }
       break;
    case GLUT_LEFT_BUTTON:
       if (state==GLUT_DOWN && !setting.mouseLeftMode)
       {
          setting.mouseX = x;
          setting.mouseY = y;
          setting.mouseLeftMode = true;
       }
       if (state==GLUT_UP &&  setting.mouseLeftMode)
       {
          setting.mouseLeftMode = false;
       }
       break;
 }
}

//BEGIN LAB10 Specific - define local axis for the spotlight and
//                       allow us to move the spot lights around
void updateWorldRotation(int xinc, int yinc)
{
 if (setting.mouseRightMode)
 {
    world.rotate(0.0f, 0.0f, -xinc*0.5);
 }
```

```
 if (setting.mouseLeftMode)
 {
    world.rotate(-yinc*0.5, xinc*0.5, 0.0f);
 }
}

void updateLocalRotation(int xinc, int yinc)
{
 if (setting.mouseRightMode)
 {
    local.rotate(0.0f, 0.0f, -xinc*0.5);
 }
 if (setting.mouseLeftMode)
 {
    local.rotate(-yinc*0.5, xinc*0.5, 0.0f);
 }
}
//END LAB10 Specific

void myMotionFunc(int x, int y)
{
 y = window.height - y;
 GLint xinc = x - setting.mouseX;
 GLint yinc = y - setting.mouseY;

 //BEGIN LAB10 Specific – define local axis for the spotlight and
 //                       allow us to move the spot lights around
 /*if(setting.mouseRightMode)
 {
    world.rotate(0.0f, 0.0f, -xinc*0.5);
 }
 if(setting.mouseLeftMode)
 {
    world.rotate(-yinc*0.5, xinc*0.5, 0.0f);
 }*/
 switch (movementType)
 {
    case moveworld: updateWorldRotation(xinc,yinc);
                    break;
    case movelocal: updateLocalRotation(xinc,yinc);
                    break;
 }
 //END LAB10 Specific

 setting.mouseX = x;
 setting.mouseY = y;
 glutPostRedisplay();
}

void myDataInit()
{
 window.title = "TCG6223 Computer Graphics";
 window.posX = 100;
 window.posY = 100;
 window.width  = 800;
 window.height = 500;

 world.rotateX  = 0.0;
 world.rotateY  = 0.0;
 world.rotateZ  = 0.0;
```

```
 world.posX     = 0.0;
 world.posY     = 0.0;
 world.posZ     = 0.0;
 world.scaleX   = 1.0;
 world.scaleY   = 1.0;
 world.scaleZ   = 1.0;

 viewer.upX     = 0.0;
 viewer.upY     = 1.0;
 viewer.upZ     = 0.0;
 viewer.zNear   = 0.1;
 viewer.zFar    = 500.0;
 viewer.fieldOfView = 60.0;
 viewer.aspectRatio = static_cast<GLdouble> (window.width) / window.height;

 setting.mouseX   = 0;
 setting.mouseY   = 0;

 setting.mouseRightMode = false;
 setting.mouseLeftMode = false;

 setting.shadingMode = true;

 //BEGIN LAB10 Specific – change the viewing/camera position/parameters
 //                       and add other parameters
 //viewer.eyeX    = 0.0;
 //viewer.eyeY    = 0.0;
 //viewer.eyeZ    = 40.0;
 //viewer.centerX = 0.0;
 //viewer.centerY = 0.0;
 //viewer.centerZ = 0.0;
 //setting.posInc   = 1.0;
 //setting.angleInc = 2.0;
 viewer.eyeX    = 0.0;
 viewer.eyeY    = 20.0;
 viewer.eyeZ    = 50.0;
 viewer.centerX = 0.0;
 viewer.centerY = 20.0;
 viewer.centerZ = 0.0;
 setting.posInc   = 10.0;
 setting.angleInc = 3.0;

 local.rotateX  = 0.0;
 local.rotateY  = 0.0;
 local.rotateZ  = 0.0;
 local.posX     = 0.0;
 local.posY     = 0.0;
 local.posZ     = 0.0;
 local.scaleX   = 1.0;
 local.scaleY   = 1.0;
 local.scaleZ   = 1.0;
 movementType = moveworld;
 spotlightsaxis.setLength(10.0f, 10.0f, 10.0f);
 spotlightsaxis.setLineWidth(3);
 spotlightsaxis.setLineStipple(1, 0xff60);
 //END LAB10 Specific
}

void myViewingInit()
{
```

```
 glMatrixMode(GL_PROJECTION);
 glLoadIdentity();
 gluPerspective(viewer.fieldOfView,
                viewer.aspectRatio,
                viewer.zNear,
                viewer.zFar);

 glMatrixMode(GL_MODELVIEW);
 glLoadIdentity();
 gluLookAt(viewer.eyeX,   viewer.eyeY,   viewer.eyeZ,
           viewer.centerX,viewer.centerY,viewer.centerZ,
           viewer.upX,    viewer.upY,    viewer.upZ );
}

//BEGIN LAB10 Specific - delete this for Lab11, lighting will be initialized
//                       in the virtualworld
/*void myLightingInit()
{
 static GLfloat  ambient[] = { 0.0f,  0.0f,  0.0f, 1.0f };
 static GLfloat  diffuse[] = { 1.0f,  1.0f,  1.0f, 1.0f };
 static GLfloat specular[] = { 1.0f,  1.0f,  1.0f, 1.0f };
 static GLfloat  specref[] = { 1.0f,  1.0f,  1.0f, 1.0f };
 static GLfloat position[] = {10.0f, 10.0f, 10.0f, 1.0f };
 short shininess = 128;

 glEnable(GL_LIGHTING);
 glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
 glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
 glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
 glLightfv(GL_LIGHT0, GL_POSITION, position);
 glEnable(GL_LIGHT0);

 glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);
 glEnable(GL_COLOR_MATERIAL);

 glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
 glMateriali(GL_FRONT, GL_SHININESS, shininess);

 glEnable(GL_NORMALIZE);
}*/
//END LAB10 Specific

void myInit()
{
 myDataInit();

 glutInitDisplayMode( GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH );
 glutInitWindowPosition(window.posX, window.posY); // Set top-left position
 glutInitWindowSize(window.width, window.height); //Set width and height
 glutCreateWindow(window.title.c_str());// Create display window

 glutDisplayFunc(myDisplayFunc);  // Specify the display callback function
 glutReshapeFunc(myReshapeFunc);
 glutKeyboardFunc(myKeyboardFunc);
 glutSpecialFunc(mySpecialFunc);
 glutMotionFunc(myMotionFunc);
 glutMouseFunc(myMouseFunc);

 glPointSize(4.0);
 glEnable(GL_DEPTH_TEST);
```

```
 glDepthFunc(GL_LESS);
 glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
 glFrontFace(GL_CCW);
 glShadeModel (GL_SMOOTH);
 glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
 glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST);

 glEnable(GL_CULL_FACE);

 myViewingInit();

 //BEGIN LAB10 Specific – lighting is now initialized within virtualworld
 //myLightingInit();
 glutFullScreen(); //set it to full screen, this is optional but nice
 //END LAB10 Specific


 myvirtualworld.init();
}

void myWelcome()
{
 cout << "*****************************************************************\n";
 cout << "*                    TCG6223 Computer Graphics                  *\n";
 cout << "*                      FIST, Multimedia University            *\n";
 cout << "*    Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>     *\n";
 cout << "*****************************************************************\n";
 cout << "| Press:                                                        |\n";
 cout << "|   <a>,<d>,<w>,<s>,<q>,<e> => move world                       |\n";
 cout << "|   <arrows>                => rotate world                     |\n";
 cout << "|   HOME                    => restore defaults                 |\n";
 cout << "|   ESC                     => exit                             |\n";
 cout << "|                                                               |\n";
 cout << "|   F1                      => toggle shading / wire-frame mode |\n";
 cout << "|   F2                      => toggle rendering of axes         |\n";
 cout << "|   F3                      => toggle lighting on / off         |\n";
 cout << "|                                                               |\n";
 cout << "| Mouse (Left Drag or Right Drag) => rotate world               |\n";
 cout << "|                                                               |\n";
 cout << "*****************************************************************\n";
 cout << "|                       H A V E   F U N  !!!                    |\n";
 cout << "*****************************************************************\n";

 //BEGIN LAB10 Specific – these keys will be defined later
 cout << "+---------------------------------------------------------------+\n";
 cout << "| ADDITIONAL KEYS FOR LAB10 :                                   |\n";
 cout << "|   1                       => 'moving world' mode              |\n";
 cout << "|   2                       => 'moving spotlights' mode         |\n";
 cout << "|   F5                      => toggle direction light           |\n";
 cout << "|   F6                      => toggle spotlight #1              |\n";
 cout << "|   F7                      => toggle spotlight #2              |\n";
 cout << "|   F8                      => toggle spotlight #3              |\n";
 cout << "|   F9                      => toggle swinging point light #1   |\n";
 cout << "|   F10                     => toggle swinging point light #2   |\n";
 cout << "|   F11                     => toggle flying point light        |\n";
 cout << "+---------------------------------------------------------------+\n";
 //END LAB10 Specific

 system("pause");  //let you see the welcome screen before going fullscreen
}
```

```
//-----------------------------------------------------------------------
int main(int argc, char **argv)
{
 glutInit(&argc, argv);

 myWelcome();

 myInit();

 glutMainLoop(); // Display everything and wait
}
//-----------------------------------------------------------------------
```

Take note that as mentioned earlier, the main modification is to allow us to define the spotlights later. Some of the modifications are done so that we will be able to move these spotlights. To achieve this, we defined additional axes and a 'local' coordinate system so that we could modify this 'local' position and orientation during run-time using mouse, thus moving and rotating the spotlights.

We have also created two type of movement, one is to move or rotate the world coordinate system and the other is to move the spotlights 'local' coordinate system.

We will later add some more user interacting using keyboard, we will add them one by one later, for an idea of what we will add later, take a look at the `myWelcome()` function.

You should take a closer look at this program; the main changes are highlighted above.

Now we are ready to create the `CGLab10and11.hpp` as below:

```
/*
 TCG6223 Computer Graphics

 CGLab10and11.hpp

 Objective: Header File for:
                Lab10 on Lighting &
                Lab11 on Textures

 Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>

 INSTRUCTIONS
 ============
 Please refer to CGLab10and11main.cpp for instructions

 CHANGE LOG
 ==========
*/

#ifndef YP_CGLAB10and11_HPP
#define YP_CGLAB10and11_HPP

#include <cmath>
#include "CGLab09.hpp"
#include "CGLabmain.hpp"
#include "CGLab10and11.hpp"
```

```
namespace CGLab10and11 {

//Make use of the MyMovingSmiley and MyFan from Lab09
using CGLab09::MyMovingSmiley;
using CGLab09::MyFan;

class MySpotLights
{
 public:
    MySpotLights();
    ~MySpotLights();

    void setupLights();
    void toggleLight(int lightno);
    void draw();
    void tickTime(long int elapseTime); //elapsetime in milisec

 private:
    GLUquadricObj *pObj;
    GLfloat rotateangle, rotatespeed;
    bool lighton[3];  //keep track if lights are on or off
};

class MySwingLights
{
 public:
    MySwingLights();
    ~MySwingLights();
    void setupLights();
    void toggleLight(int lightno);
    void draw();
    void tickTime(long int elapseTime); //elapsetime in milisec

 private:
    GLUquadricObj *pObj;
    GLfloat length; //length of the pendulum
    GLfloat horizDisp1, horizDisp2; //horizontal displacement
    long int timestart;

    bool lighton[2];  //keep track if lights are on or off
};

class MyVirtualWorld
{
 public:
    MyVirtualWorld();
    ~MyVirtualWorld();

    void init();
    void setupLights();
    void toggleLight(int lightno);
    void draw();

    void drawSpotLights();

    void tickTime()
    {
     timenew    = glutGet(GLUT_ELAPSED_TIME);
     elapseTime = timenew - timeold;
```

```
     timeold    = timenew;

     mymovingsmiley.tickTime(elapseTime);
              myfan.tickTime(elapseTime);
       myspotlights.tickTime(elapseTime);
       myswinglights.tickTime(elapseTime);

    }

 private:
    long int timeold, timenew, elapseTime;

    MyMovingSmiley mymovingsmiley;
    MyFan myfan;
    MySpotLights myspotlights;
    MySwingLights myswinglights;

    GLfloat xmin, zmin, xmax, zmax, height,
            xdim, ydim, zdim;

    void drawRoof();
    void drawFloor();
    void drawWalls();
    void drawFourTeapot();

    bool lighton[7];  //keep track if lights are on or off
};

}; //end of namespace CGLab10and11

#endif //YP_CGLAB10and11_HPP
```

The `CGLab10and11.cpp` is as below:

```
/*
 TCG6223 Computer Graphics

 CGLab10and11.cpp

 Objective: Lab10 on Lighting &
            Lab11 on Textures

 Copyright (C) 2011 by Ya-Ping Wong <ypwong@mmu.edu.my>

 INSTRUCTIONS
 ============
 * Please refer to CGLabmain.cpp for instructions

 CHANGE LOG
 ==========
*/

#include <GL/glut.h>
#include "CGLab10and11.hpp"

using namespace CGLab10and11;

MyVirtualWorld::MyVirtualWorld()
{
```

```
 xmin = zmin =-50.0f;
 xmax = zmax = 50.0f;
 height = 50.0f;
 xdim = xmax-xmin;
 zdim = zmax-zmin;
 ydim=height;
}

MyVirtualWorld::~MyVirtualWorld()
{
}

void MyVirtualWorld::init()
{
 setupLights();
 timeold = glutGet(GLUT_ELAPSED_TIME);
}

void MyVirtualWorld::setupLights()
{
 //to be filled later.....
}

void MyVirtualWorld::toggleLight(int lightno)
{
//to be filled later.....
}


void MyVirtualWorld::draw()
{
 glPushMatrix();
    mymovingsmiley.draw();
    myfan.draw();
    drawFourTeapot();
    glPushMatrix();
       glTranslatef(0.0f, -0.01f, 0.0f);
       drawFloor();
       drawRoof();
       drawWalls();
    glPopMatrix();
    glPushMatrix();
       glTranslatef(0.0f, height, 0.0f);
       myswinglights.draw();
    glPopMatrix();
glPopMatrix();
}

void MyVirtualWorld::drawSpotLights()
{
 myspotlights.draw();
}

void MyVirtualWorld::drawFloor()
{
 glPushMatrix();
    GLfloat tilesize=0.25f;
    for (GLfloat z=zmin; z<zmax; z+=tilesize)
    {
       for (GLfloat x=xmin; x<xmax; x+=tilesize)
```

```
       {
          glColor3fv( mywhite );
          glBegin(GL_QUADS);
             glNormal3f(0.0f, 1.0f, 0.0f);
             glVertex3f(x         , 0.0f, z);
             glVertex3f(x         , 0.0f, z+tilesize);
             glVertex3f(x+tilesize, 0.0f, z+tilesize);
             glVertex3f(x+tilesize, 0.0f, z);
          glEnd();
       }
    }
 glPopMatrix();
}

void MyVirtualWorld::drawRoof()
{
 static GLfloat* color[] = { myblack, mywhite };

 glPushMatrix();
    GLfloat xstep=5.0f;
    GLfloat zstep=5.0f;
    GLfloat tilesize=2.5f;

    int startcolorno = 0;
    for (GLfloat z=zmin; z<zmax; z+=zstep)
    {
       int colorno = startcolorno;
       for (GLfloat x=xmin; x<xmax; x+=xstep)
       {
          glColor3fv( color[colorno] );
          GLfloat xmax = x+xstep;
          GLfloat zmax = z+zstep;
          for (GLfloat xx=x; xx<xmax; xx+=tilesize)
          for (GLfloat zz=z; zz<zmax; zz+=tilesize)
          {
             glBegin(GL_QUADS);
                glNormal3f(0.0f, -1.0f, 0.0f);
                glVertex3f(xx         , height, zz);
                glVertex3f(xx+tilesize, height, zz);
                glVertex3f(xx+tilesize, height, zz+tilesize);
                glVertex3f(xx         , height, zz+tilesize);
             glEnd();
          }
          ++colorno;
          if (colorno>1) colorno=0;
       }
       ++startcolorno;
       if (startcolorno>1) startcolorno=0;
    }
 glPopMatrix();

}

void MyVirtualWorld::drawWalls()
{
 static GLfloat* color[] =
    { myblack, myred, myyellow, mygreen,
      mycyan, myblue, mymagenta, mywhite };
    //take note, the myred, myblue etc. are defined in CGLabmain.hpp
```

```
glPushMatrix();
   GLfloat ystep=5.0f;
   GLfloat zstep=5.0f;
   GLfloat tilesize=2.5f;

   GLfloat xdim=xmax-xmin;
   GLfloat ydim=height;
   GLfloat zdim=zmax-zmin;
   GLfloat ymin=0.0f;
   GLfloat ymax=height;

   int startcolorno = 0;
   for (GLfloat z=zmin; z<zmax; z+=zstep)
   {
      int colorno = startcolorno;
      for (GLfloat y=0.0f; y<height; y+=ystep)
      {
         glColor3fv( color[colorno] );

         GLfloat ymax = y+ystep;
         GLfloat zmax = z+zstep;
         for (GLfloat yy=y; yy<ymax; yy+=tilesize)
         for (GLfloat zz=z; zz<zmax; zz+=tilesize)
         {
            glBegin(GL_QUADS);
               glNormal3f(-1.0f, 0.0f, 0.0f);
               glVertex3f(xmax, yy         , zz);
               glVertex3f(xmax, yy         , zz+tilesize);
               glVertex3f(xmax, yy+tilesize, zz+tilesize);
               glVertex3f(xmax, yy+tilesize, zz);

               glNormal3f(1.0f, 0.0f, 0.0f);
               glVertex3f(xmin, yy         , zz);
               glVertex3f(xmin, yy+tilesize, zz);
               glVertex3f(xmin, yy+tilesize, zz+tilesize);
               glVertex3f(xmin, yy         , zz+tilesize);
            glEnd();
         }
         ++colorno;
         if (colorno>7) colorno=0;
      }
      ++startcolorno;
      if (startcolorno>7) startcolorno=0;
   }

   GLfloat xstep=5.0f;

   startcolorno = 0;
   for (GLfloat x=xmin; x<xmax; x+=xstep)
   {
      int colorno = startcolorno;
      for (GLfloat y=0.0f; y<height; y+=ystep)
      {
         glColor3fv( color[colorno] );
         GLfloat ymax = y+ystep;
         GLfloat xmax = x+xstep;
         for (GLfloat yy=y; yy<ymax; yy+=tilesize)
         for (GLfloat xx=x; xx<xmax; xx+=tilesize)
```

```
          {
             glBegin(GL_QUADS);
                glNormal3f(0.0f, 0.0f, 1.0f);
                glVertex3f(xx         , yy         , zmin);
                glVertex3f(xx+tilesize, yy         , zmin);
                glVertex3f(xx+tilesize, yy+tilesize, zmin);
                glVertex3f(xx         , yy+tilesize, zmin);

                glNormal3f(0.0f, 0.0f, -1.0f);
                glVertex3f(xx         , yy         , zmax);
                glVertex3f(xx         , yy+tilesize, zmax);
                glVertex3f(xx+tilesize, yy+tilesize, zmax);
                glVertex3f(xx+tilesize, yy         , zmax);
             glEnd();
          }
          ++colorno;
          if (colorno>7) colorno=0;
       }
       ++startcolorno;
       if (startcolorno>7) startcolorno=0;
    }
 glPopMatrix();
}

void MyVirtualWorld::drawFourTeapot()
{
 //for some reason, glutSolidTeapot are using
 //   clock-wise winding, thus must set this for them
 //   to be properly rendered
 glFrontFace(GL_CW);
 GLfloat dx = xmax-xmin-20.0f;
 GLfloat dz = zmax-zmin-20.0f;
 glPushMatrix();
    glTranslatef(0.0f, 5.0f, 0.0f);

    glTranslatef(xmax-10.0f, 0.0f, zmax-10.0f);
    glColor3f(1.0f, 0.0f, 0.0f );
    glutSolidTeapot(5);

    glTranslatef(-dx, 0.0f, 0.0f);
    glColor3f(0.0f, 1.0f, 0.0f );
    glutSolidTeapot(5);
    glTranslatef(0.0f, 0.0f, -dz);
    glColor3f(0.0f, 0.0f, 1.0f );
    glutSolidTeapot(5);
    glTranslatef(dx, 0.0f, 0.0f);
    glColor3f(0.0f, 1.0f, 1.0f );
    glutSolidTeapot(5);
 glPopMatrix();
 //set back to Counter-Clock Winding
 glFrontFace(GL_CCW);
}

MySpotLights::MySpotLights()
{
 pObj = gluNewQuadric();
 rotateangle= 0.0f;
 rotatespeed = 360.0;  //degree per sec
 gluQuadricNormals(pObj, GLU_SMOOTH);
```

```
}

MySpotLights::~MySpotLights()
{
}

void MySpotLights::setupLights()
{
 //to be filled later.....
}

void MySpotLights::toggleLight(int lightno)
{
 //to be filled later.....
}

void MySpotLights::draw()
{
 GLboolean cullingIsOn;
 glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
 glDisable(GL_CULL_FACE);

 GLfloat radius=9.0f;

 glPushMatrix();
    glRotatef(90.0f, 1.0f, 0.0f, 0.0f);
    glRotatef(rotateangle, 0.0f, 0.0f, 1.0f);
    glColor3f(1.0f, 0.5f, 0.3f);
    gluDisk(pObj,radius-1.0f, radius+1.0f, 20, 5);
    glPushMatrix();
       glTranslatef(0.0f, 0.0f, -0.1f);
       glRotatef(180.0f, 1.0f, 0.0f, 0.0f);
       gluDisk(pObj,radius-1.0f, radius+1.0f, 20, 5);
    glPopMatrix();

    glTranslatef(radius, 0.0f, 0.0f);
    glColor3f(1.0f, 0.0f, 0.0f);
    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);

    GLfloat dx=radius*0.5;   // = radius*cos(M_PI/3);
    GLfloat dy=radius*0.8660;// = radius*sin(M_PI/3);

    glTranslatef(-radius-dx, dy, 0.0f);
    glColor3f(0.0f, 1.0f, 0.0f);
    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);

    glTranslatef(0.0f, -2*dy, 0.0f);
    glColor3f(0.0f, 0.0f, 1.0f);
    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);

 glPopMatrix();
 if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
}

void MySpotLights::tickTime(long int elapseTime) //elapsetime in milisec
{
 rotateangle += elapseTime * rotatespeed / 1000.0;

 if (rotateangle>=360)
 {
```

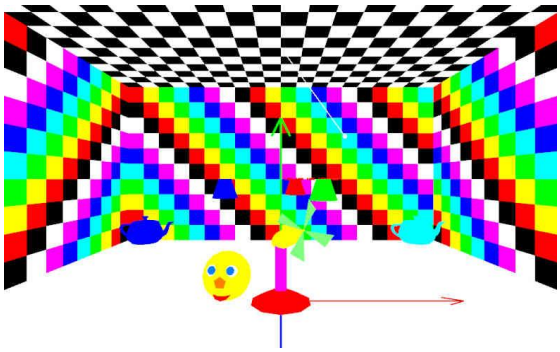
```
    rotateangle = 360 - rotateangle;
 }
};

MySwingLights::MySwingLights()
{
 pObj = gluNewQuadric();
 gluQuadricNormals(pObj, GLU_SMOOTH);
 length = 25; //length of the pendulum
 timestart = glutGet(GLUT_ELAPSED_TIME);
}

MySwingLights::~MySwingLights()
{

}

void MySwingLights::setupLights()
{
 //to be filled up later...
}

void MySwingLights::toggleLight(int lightno)
{
 //to be filled up later...
}

void MySwingLights::draw()
{
float degreePerRad = 180/M_PI;

 glPushMatrix();
    glTranslatef(0.0f, 0.0f, -20.0f);
    glRotatef(90.0f, 1.0f, 0.0f, 0.0f);
    glRotatef(asin(horizDisp1/length)*degreePerRad, 0.0f, 1.0f, 0.0f);

    glColor3f(1.0f, 1.0f, 1.0f);
    gluCylinder(pObj, 0.1f, 0.1f, length, 10, 10);
    glTranslatef(0.0f, 0.0f, length);
    gluSphere(pObj, 0.5, 10, 10);
glPopMatrix();

 glPushMatrix();
    glTranslatef(0.0f, 0.0f, 20.0f);
    glRotatef(90.0f, 1.0f, 0.0f, 0.0f);
    glRotatef(asin(horizDisp2/length)*degreePerRad, 0.0f, 1.0f, 0.0f);

    glColor3f(1.0f, 1.0f, 1.0f);
    gluCylinder(pObj, 0.1f, 0.1f, length, 10, 10);
    glTranslatef(0.0f, 0.0f, length);
    gluSphere(pObj, 0.5, 10, 10);
glPopMatrix();
}

void MySwingLights::tickTime(long int elapseTime) //elapsetime in milisec
{
 //we ignore the elapseTime passed to this function,
 //   because we are calculating the position based on
 //   the time since the pendulum starts.
 //   This is another way to do timing for animation
```

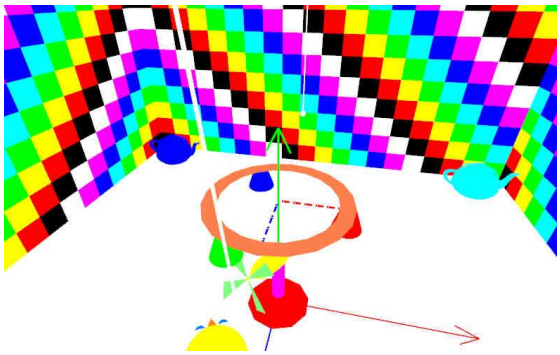
```
 float currentTime = timestart - glutGet(GLUT_ELAPSED_TIME);
 float currentTimeInSec = currentTime / 1000.0;

 float D = 25;  //D is max horizontal displacement
 float g= 50.0; //gravity, bigger value to swing faster
 float freq = sqrt( g/length ); //frequency of pendulum
 float k = freq * currentTimeInSec;

 //This equation is meant for pendulum of small angle
 //we use this as approximation though our angle is big
 horizDisp1 = D*sin( k - M_PI );
 horizDisp2 = D*sin( k );
}
```

Take special note on the highlighted part and also take note of the comments below:

- We have created above all the necessary classes and function to attach our lights to the virtual world and the models, right now, none of the lights have been created nor setup yet, we will do that later.
- **IMPORTANT : In order for lighting to work correctly, all the surface normals must be specified correctly as shown above.**

Now run the program and you will see the output (with animation) below:





Take

Take note of the comments below:

- Take note that the above shading is just flat shading and lights are not created nor activated yet.
- Try dragging the mouse while pressing the left button; this will rotate the world relative to the world's current x-axis and y-axis.
- Try dragging the mouse while pressing the right button; this will rotate the world relative to the world's current z-axis.
- You may also press the 'a', 's', 'd', 'w', 'q', 'e' keys to move the world relative to the world coordinate system.
- Please take note the movement above are relative to the world coordinate system, NOT relative to the screen coordinates.
- The above is the "move world" movement mode, there is another movement mode which is the "move spot lights" mode and can be activated by pressing the '2' key of the keyboards. To change back to "move world" mode, you need to press the '1' key. Try to move the spot lights structure (the light is not created yet) around.

- Press F1 to toggle the shading and wire-frame mode. Take note that each tile on the walls and roof are formed by more than 1 quad.
- Press F2 to toggle of the rendering of the axes.

Try to understand the program and discuss as much as you can before you proceed in the following sections.

Task 1B: Setting up lighting mode

To enable lighting mode, add the following code to the `void MyVirtualWorld::setupLights()` function as below:

```
void MyVirtualWorld::setupLights()
{
 glEnable(GL_LIGHTING);

 //GL_COLOR_MATERIAL
 //  * relevant only if lighting is enabled
 //  * disabled by default
 //  * if enabled, glColor*(...) is in effect to change the color
 //                tracked by glColorMaterial
 //                (meaning that in our case here, glColor*(...) affect
 //                the diffuse color of the frant face)
 //  * if disbled, glMaterial*(...) is in effect to change the color
 //                glColor*(...) will not work!
 //glColor*(...) always in effect if lighting is not enabled
 glColorMaterial(GL_FRONT, GL_DIFFUSE);
 glEnable(GL_COLOR_MATERIAL);

 //ensure unit vectors remain unit vectors after
 //  modelview scaling
 glEnable(GL_NORMALIZE);
}
```

Take note that `MyVirtualWorld::setupLights()` is being called by `MyVirtualWorld::init()`, which in turn is called by `MyInit()` function of `CGLab10and11main.cpp` program.

Take note of the comments regarding the **`GL_COLOR_MATERIAL`** mode; please always refer to the Blue Book (OpenGL Reference Manual) to know what all the OpenGL functions mean. The OpenGL blue book is available at:

http://www.opengl.org/documentation/blue_book/

Run the program and what do you see?

Oouch.... almost total darkness!! ... except the axes where we deliberately disable the lighting in our code when rendering those axes. If you press F2 to turn off the axes, then it will be in total darkness. Remember, since Lab 01, you can press F3 to toggle the lighting on/off state, if you do that you will be able to see the world without the lighting effect.

Why total darkness?

... because lighting is enable but there is no lights defined so far!!

Task 1C: Setting up a light

Now we are ready to define one light for our virtual world!

Add the highlighted lines below to the `MyVirtualWorld::setupLights()` function, these code is to define a very dim white light (i.e. grey light) in our virtual world.

```
void MyVirtualWorld::setupLights()
{
 ...
 ...
 myspotlights.setupLights();
 myswinglights.setupLights();

 //define the color of light, i.e. LIGHT0
 GLfloat mycolor[] = { 0.15, 0.15, 0.15};
 glLightfv(GL_LIGHT0, GL_DIFFUSE, mycolor);

 //enable the light, i.e. LIGHT0
 glEnable(GL_LIGHT0);
}
```

Here we declared the light to be GL_LIGHT0. Depending on implementation, OpenGL support at least 8 lights, i.e. GL_LIGHT0, GL_LIGHT1, ..., GL_LIGHT7.

The next thing to do is to define the light position or direction, to do that you need to add the code below in the beginning of the `MyVirtualWorld::draw()` function.

```
void MyVirtualWorld::draw()
{
 //define the GL_POSITION of the light, since the 4th number
 //   is 0.0 rather than 1.0, this light is a directional light
 //Take note that the GL_POSITION should be specified in the
 //   modeling code (here) so that it will be relative to the
 //   current coordinate system.
 GLfloat position[] = {1.0f, 1.0f, 0.0f, 0.0f};
 glLightfv(GL_LIGHT0, GL_POSITION, position);
 ...
 ...
}
```

Take note that normally, we will define a point light as below:

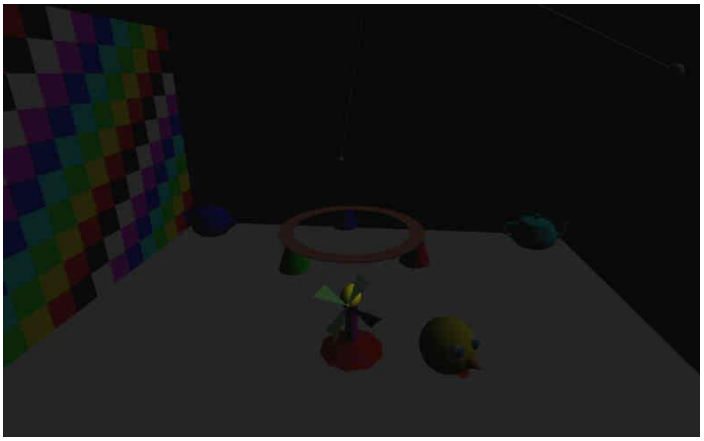
```
GLfloat position[] = {1.0f, 1.0f, 0.0f, 1.0f};  //4th value is 1.0
glLightfv(GL_LIGHT0, GL_POSITION, position);
```

The `position[]` array stores the position of the light in the current coordinate system IF the 4th number of the homogenous coordinate is 1.0. IF this number is 0.0 as shown in our example which

we have just created, then this is to indicate the direction of the light (the direction of which the light is shining from). In this case, the light is called a "directional light" to simulate light from very far away such as the Sun.

IMPORTANT: Take note you MUST specify the position or direction relative to the current coordinate system and not the world coordinate system. In another word, you must specify the position or direction of the light along with your modelling code, not during initialization.

Now, run the program and you will see a dimly lit virtual world as below:



Try changing the direction of the light to see its effect.

Task 2: Creating more lights *(20 minutes)*

The directional light that we have earlier is a very simple light with no associated geometrical models attached to it, thus we simply defined it within the `MyVirtualWorld` class itself.

However, for the lights we will be creating after this below, the lights are attached to some objects and contain positions information as well; thus they are more complicated, so it is better to use other classes to define them.

In the above, we have already defined the `MySpotLights` and `MySwingLights` classes; we shall attach our lights to these geometrical models.

Task 2A: Creating spotlights

Since the `void myInit()` function from the main program will call the `void MyVirtualWorld::setupLights()` to setup the lights, we shall simply let this `void MyVirtualWorld::setupLights()` function to call the `setupLights()` function of our lights, this is a good way to modularize our program.

Add the highlighted line at the end of the `MyVirtualWorld::setupLights()` function as below:

```
void MyVirtualWorld::setupLights()
{
 ...
 ...
 myspotlights.setupLights();
}
```

Take note that `myspotlights` is an instance of the `MySpotLights` class. The `myspotlights` is a private member of the `MyVirtualWorld` class.

The next thing to do is to setup the lights by filling up the `MySpotLights::setupLights()` function as highlighted below:

```
void MySpotLights::setupLights()
{
glLightfv(GL_LIGHT1, GL_AMBIENT, myred);
 glLightfv(GL_LIGHT1, GL_DIFFUSE, myred);
 glLightfv(GL_LIGHT1, GL_SPECULAR, myred);
 glLightf (GL_LIGHT1, GL_SPOT_CUTOFF, 35);
 glLightf (GL_LIGHT1, GL_SPOT_EXPONENT, 2.0);
 glLightf (GL_LIGHT1, GL_CONSTANT_ATTENUATION, 1.0);
 glLightf (GL_LIGHT1, GL_LINEAR_ATTENUATION, 0.04);

 glLightfv(GL_LIGHT2, GL_AMBIENT, mygreen);
 glLightfv(GL_LIGHT2, GL_DIFFUSE, mygreen);
 glLightfv(GL_LIGHT2, GL_SPECULAR, mygreen);
 glLightf (GL_LIGHT2, GL_SPOT_CUTOFF, 35);
 glLightf (GL_LIGHT2, GL_SPOT_EXPONENT, 2.0);
 glLightf (GL_LIGHT2, GL_CONSTANT_ATTENUATION, 1.0);
 glLightf (GL_LIGHT2, GL_LINEAR_ATTENUATION, 0.04);

 glLightfv(GL_LIGHT3, GL_AMBIENT, myblue);
 glLightfv(GL_LIGHT3, GL_DIFFUSE, myblue);
 glLightfv(GL_LIGHT3, GL_SPECULAR, myblue);
 glLightf (GL_LIGHT3, GL_SPOT_CUTOFF, 35);
 glLightf (GL_LIGHT3, GL_SPOT_EXPONENT, 2.0);
 glLightf (GL_LIGHT3, GL_CONSTANT_ATTENUATION, 1.0);
 glLightf (GL_LIGHT3, GL_LINEAR_ATTENUATION, 0.04);

 glEnable(GL_LIGHT1);
 glEnable(GL_LIGHT2);
 glEnable(GL_LIGHT3);
}
```

Here we are defining GL_LIGHT1., GL_LIGHT2, GL_LIGHT3 to be spot lights.

Characteristic of spotlights is that they have position, direction and most importantly the spot cutoff angle (in degree). You may refer to the Blue Book (you should know which book I meant by now!) for what these parameters meant, however, another good way to learn is to try to modify these values and see the effect. We can also specify how the intensity of the spotlights is spread out from the center by specifying the "spot exponent" of the lights. The best way to know what these mean is to change the values and see the effect.

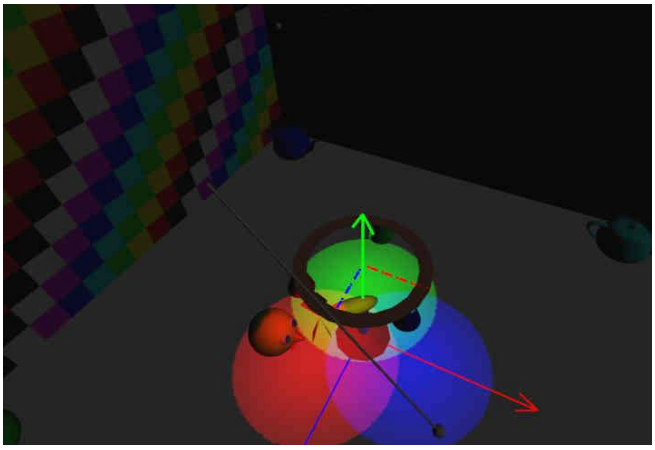
All lights with position can also have attenuation associated with them; attenuation specifies how the intensity of the light diminishes with distance. **BE CAREFUL, not to put a value that is too high!!**

IMPORTANT: As mentioned earlier, you must NOT specify position and directions of the lights in the setup function here!! Do it in the modeling code (i.e. to specify them in the model coordinate system) !!

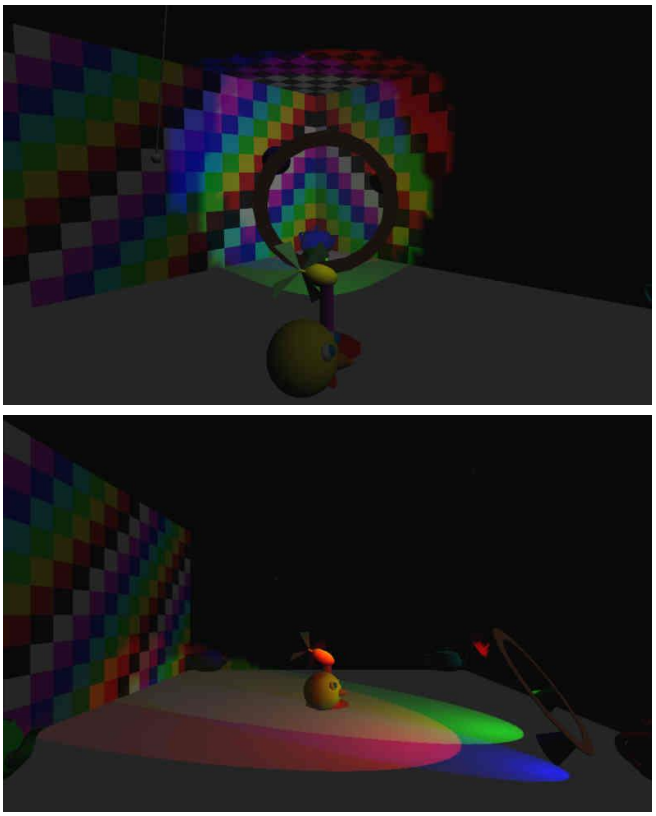
Our next task is to specify the positions and directions of the spotlights in the model themselves. We will normally just specify the lights positions and/or directions right before or after where the geometrical models are specified. To do that, insert the highlighted code into the `MySpotLights::draw()` function as shown below:

```
void MySpotLights::draw()
{
  ...
  ...
  static GLfloat position[] = {0.0f, 0.0f, 0.0f, 1.0f };
  static GLfloat direction[] = {0.0f, 0.0f, 1.0f, 1.0f };

  ...
  ...
   gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
   glLightfv(GL_LIGHT1, GL_POSITION, position);
   glLightfv(GL_LIGHT1, GL_SPOT_DIRECTION, direction);

  ...
  ...
   gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
   glLightfv(GL_LIGHT2, GL_POSITION, position);
   glLightfv(GL_LIGHT2, GL_SPOT_DIRECTION, direction);

  ...
  ...
   gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
   glLightfv(GL_LIGHT3, GL_POSITION, position);
   glLightfv(GL_LIGHT3, GL_SPOT_DIRECTION, direction);

  ...
  ...
}
```

Run the program and you should see the output below:



Try moving and rotating the spotlights (Press '2' to go into "move spot lights" mode), some sample output is shown below:



Task 2B: Creating point lights

Creating point lights is actually easier than creating spotlights. Nevertheless, we will continue by adding two point lights into our virtual world. The point lights would be attached to the two swinging structure (pendulum) in the virtual world that we have already created.

The process is very similar with the task above, the modification is as highlighted below:

```
void MyVirtualWorld::setupLights()
{
 ...
 ...
 myspotlights.setupLights();
 myswinglights.setupLights();
}
```

Take note that `myswinglights` is an instance of the `MySwingLights` class. The `myswinglights` is a private member of the `MyVirtualWorld` class.

The next thing to do is to setup the lights as below:

```
void MySwingLights::setupLights()
{
 glLightfv(GL_LIGHT4, GL_AMBIENT, mywhite);
 glLightfv(GL_LIGHT4, GL_DIFFUSE, mywhite);
 glLightfv(GL_LIGHT4, GL_SPECULAR, mywhite);
 glLightf (GL_LIGHT4, GL_CONSTANT_ATTENUATION, 1.0);
 glLightf (GL_LIGHT4, GL_LINEAR_ATTENUATION, 0.2);

 glLightfv(GL_LIGHT5, GL_AMBIENT, mywhite);
 glLightfv(GL_LIGHT5, GL_DIFFUSE, mywhite);
 glLightfv(GL_LIGHT5, GL_SPECULAR, mywhite);
 glLightf (GL_LIGHT5, GL_CONSTANT_ATTENUATION, 1.0);
 glLightf (GL_LIGHT5, GL_LINEAR_ATTENUATION, 0.2);

 glEnable(GL_LIGHT4);
 glEnable(GL_LIGHT5);
}
```

Here we are defining GL_LIGHT4 and GL_LIGHT5 to be point lights.

To get good results, you must not use an attenuation values that are too big or too small. Values that are too big will make your virtual world very dark. Values that are too small will make your virtual world to be too bright to be interesting.

Generally, you must use small attenuation values if your virtual world is small. Large attenuation values are used for big virtual world, such as modeling an outdoor world.

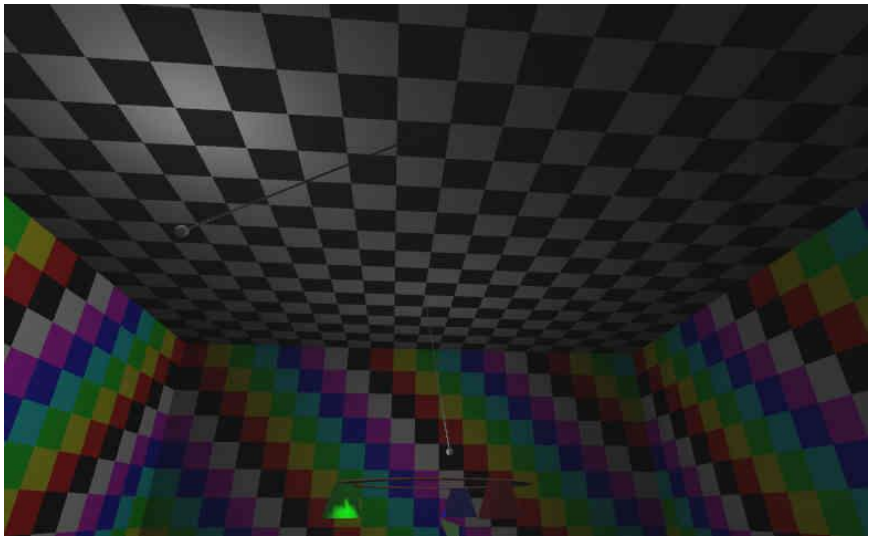
For really big world, you could use another attenuation value, which is **GL_QUADRATIC_ATTENUATION**. Generally, if you use higher order attenuation value, the lower one would not have much effect.

IMPORTANT: As mentioned earlier, you must NOT specify position in the setup function here!!

Our next task is to specify the positions of the point lights in the model themselves; in this case, we will attach them to the swinging structure.

```
void MySwingLights::draw()
{
    ...
    static GLfloat position[] = {0.0f, 0.0f, 0.0f, 1.0f };

    ...
    gluSphere(pObj, 0.5, 10, 10);
    glLightfv(GL_LIGHT4, GL_POSITION, position);

    ...
    gluSphere(pObj, 0.5, 10, 10);
    glLightfv(GL_LIGHT5, GL_POSITION, position);

    ...
}
```

Run the program and you should see the output below:



Task 3: Turning on and off lights *(10 minutes)*

We have in total 6 lights so far, they are:
- 1 directional light of `myvirtualworld`
- 3 spot lights of `myspotlights`
- 2 point lights of `myswinglights`

We will now modify the code so that we can interactively turning on or off each of the lights individually.

Modify the functions below by adding additional lines to the end of each of the functions as shown:

```
void MyVirtualWorld::setupLights()
{
 ...

 for (int i=0; i<6; ++i)
   lighton[i] = true;
}

void MySpotLights::setupLights()
{
 ...

 for (int i=0; i<3; ++i)
   lighton[i] = true;
}

void MySwingLights::setupLights()
{
 ...

 for (int i=0; i<2; ++i)
   lighton[i] = true;
}
```

As declared in the class definition of the `MyVirtualWorld`, `MySpotLights` and `MySwingLights` classes, each of these classes keep track of the light on or off status of their respective lights. This is achieved by the array of boolean values `lighton` within each class.

As shown above, `MySpotLights` is keeping track of its 3 lights. Likewise, `MySwingLights` is keeping track of its 2 lights. Since both `MySpotLights` and `MySwingLights` are contained inside `MyVirtualWorld`, they will be tracked by the main program via `MyVirtualWorld` class. Thus, `MyVirtualWorld` will have to track these 5 lights on top of its own 1 directional light.

We now modify the program by adding the highlighted lines into the appropriate existing functions as shown below:

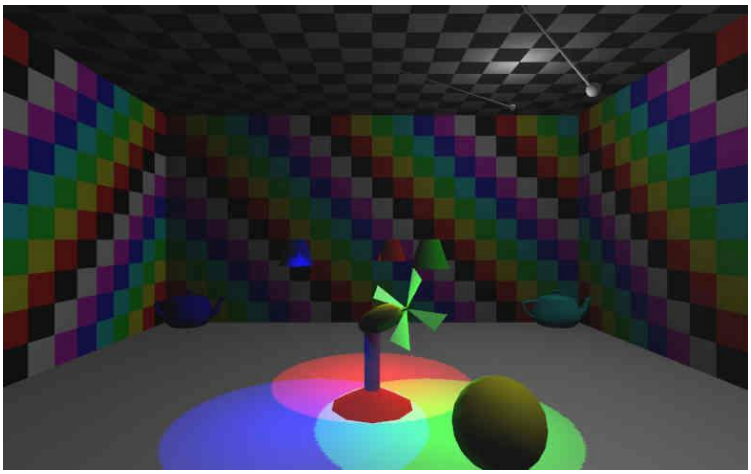
```
//light 0 is a directional light
//lights 1,2,3 are lights 0,1,2 of spotlights
//lights 4,5 are lights 0,1 of swinglights
void MyVirtualWorld::toggleLight(int lightno)
{
 if (lightno==0)
 {
    lighton[0] = !lighton[0];
    if (lighton[0])
       glEnable( GL_LIGHT0 );
    else
       glDisable( GL_LIGHT0 );

 }
 else if (lightno>=1 && lightno<=3)
 {
    myspotlights.toggleLight(lightno-1);
 }
 else if (lightno>=4 && lightno<=5)
 {
    myswinglights.toggleLight(lightno-4);
 }
}

void MySpotLights::toggleLight(int lightno)
{
 static GLenum tag[] = {GL_LIGHT1, GL_LIGHT2, GL_LIGHT3};

 lighton[lightno] = !lighton[lightno];
 if (lighton[lightno])
    glEnable( tag[lightno] );
 else
    glDisable( tag[lightno] );
}

void MySwingLights::toggleLight(int lightno)
{
 static GLenum tag[] = {GL_LIGHT4, GL_LIGHT5};

 lighton[lightno] = !lighton[lightno];
 if (lighton[lightno])
    glEnable( tag[lightno] );
 else
    glDisable( tag[lightno] );
}
```

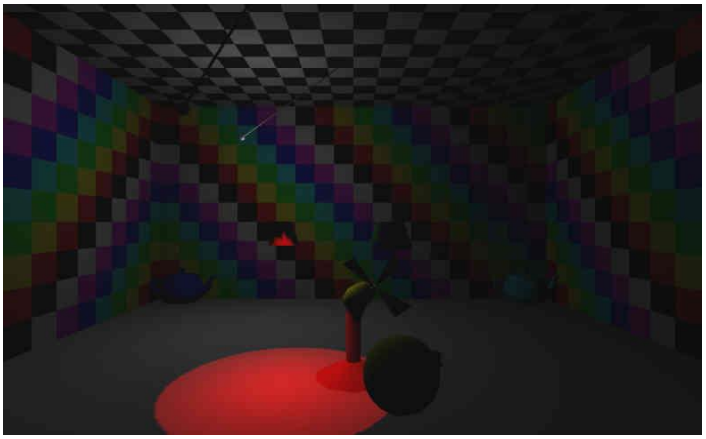
The next thing to do is to modify the special keyboard callback function as below:

```
void mySpecialFunc(int key, int x, int y)
{
 ...
 ...
 ...
  switch (key)
  {
     ...
     ...

    //BEGIN LAB10 Specific
      case GLUT_KEY_F5    : myvirtualworld.toggleLight(0);
                            break;
      case GLUT_KEY_F6    : myvirtualworld.toggleLight(1);
                            break;
      case GLUT_KEY_F7    : myvirtualworld.toggleLight(2);
                            break;
      case GLUT_KEY_F8    : myvirtualworld.toggleLight(3);
                            break;
      case GLUT_KEY_F9    : myvirtualworld.toggleLight(4);
                            break;
      case GLUT_KEY_F10   : myvirtualworld.toggleLight(5);
                            break;
    //END LAB10 Specific
 ...
 ...
}
```

Now, run the program and you can turn on or off each of the light using the F5 to F10 function keys!!

Some sample output is shown here:

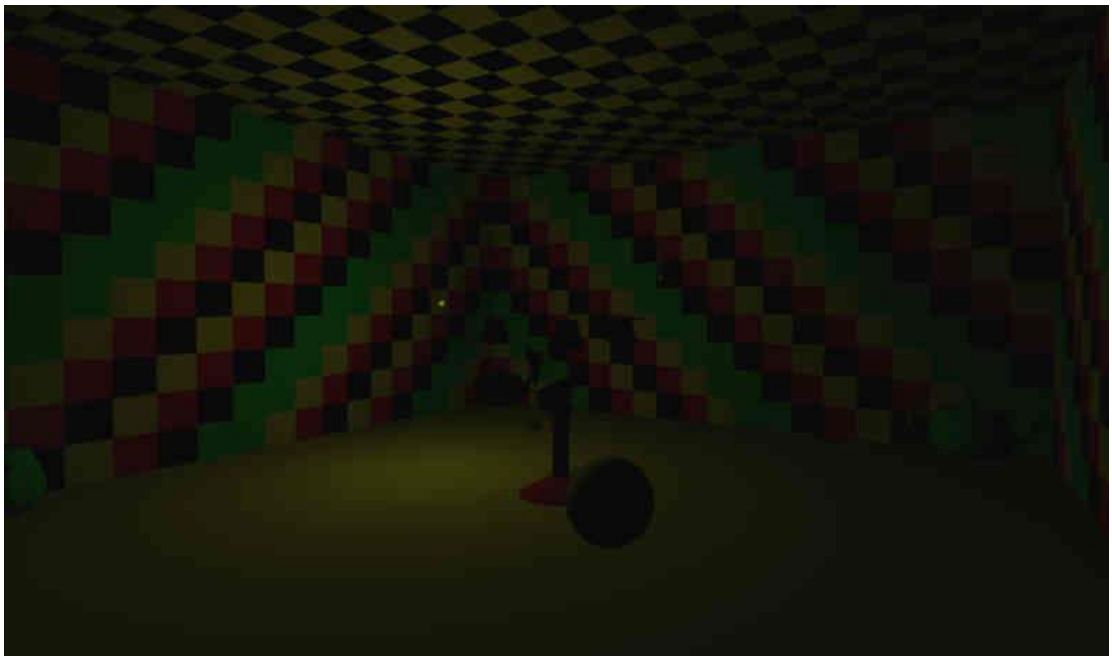




Task 4: Practice – Flying Light (Firefly)! *(30 minutes)*

Create a new light called `MyFlyingLights` and let it fly through the virtual world either in a certain pattern or randomly. You must be able to turn it on or off by pressing F11 function key.

Sample output of a yellow flying random light (with all other lights turned off):



Special Note: Lab 11 will be based on this lab, thus you must at least have completed Task 1 and Task 2 here before you could proceed to do Lab 11. Lab 11 will be on textures.

HAVE FUN, LET THERE BE LIGHTS!!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

1. COMPLETE all the exercises in this lab!

2. DO the extra exercises below (if there are any)!

3. READ the lab instructions!

4. PREPARE files for next lab!

1. Complete the exercises if you have not completed them. Be creative and try out ideas to make nice virtual world and light out your world!

2. Discussion:

When you run the program, press F1 to go into wireframe shading mode, you will notice that the floor of the virtual world is formed by many small quads. Since the floor is just one huge rectangle (quad), why can't we model it as one single quad or just a few bigger quads??

Let's take a closer look at the `MyVirtualWorld::drawFloor()` function as below:

```
void MyVirtualWorld::drawFloor()
{
 glPushMatrix();
    GLfloat tilesize=0.25f;
    for (GLfloat z=zmin; z<zmax; z+=tilesize)
    {
       for (GLfloat x=xmin; x<xmax; x+=tilesize)
       {
          glColor3fv( mywhite );
          glBegin(GL_QUADS);
             glNormal3f(0.0f, 1.0f, 0.0f);
             glVertex3f(x          , 0.0f, z);
             glVertex3f(x          , 0.0f, z+tilesize);
             glVertex3f(x+tilesize, 0.0f, z+tilesize);
             glVertex3f(x+tilesize, 0.0f, z);
          glEnd();
       }
    }
 glPopMatrix();
}
```

The size of the tiles (quads) is controlled by the variable `tilesize`, the bigger this value, the less polygon we will have resulting the rendering to be faster. Try changing the value to a bigger value such as 1.0f or 4.0f, what do you observed about the spotlights effect?

The problem you observed is a well-known problem associated with the default Gouraud shading method!

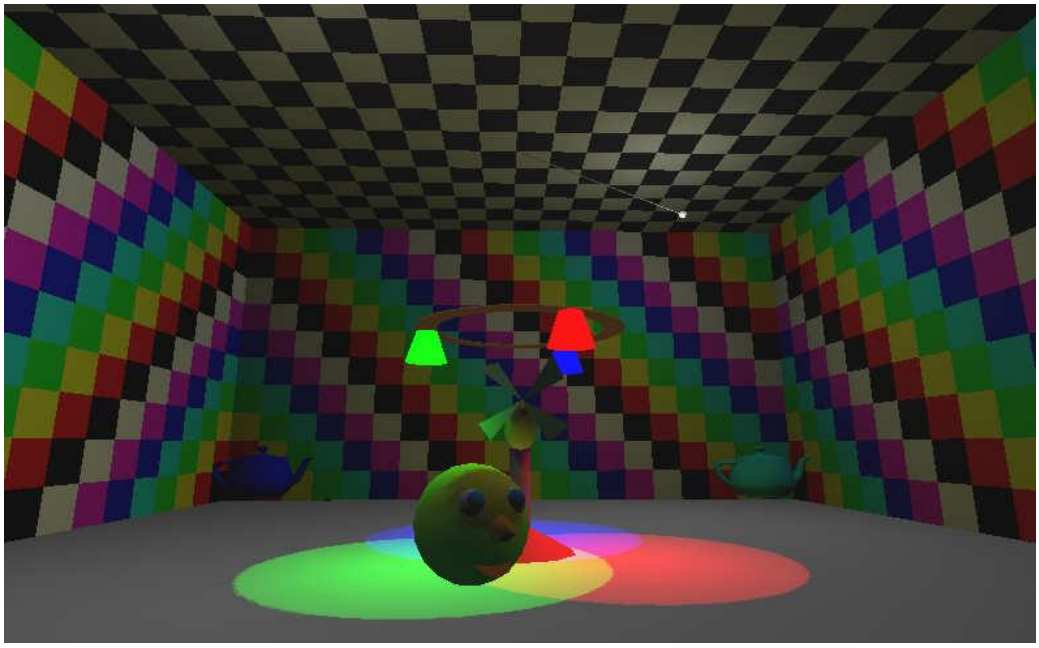
3. To make a light more realistic, we would want the light geometry to have emmisive color when it is turns on, to do this, replace the `MySpotLights::draw()` function with the one below:

```
void MySpotLights::draw()
{
 GLboolean cullingIsOn;
 glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
 glDisable(GL_CULL_FACE);

 static GLfloat position[] = {0.0f, 0.0f, 0.0f, 1.0f };
 static GLfloat direction[] = {0.0f, 0.0f, 1.0f, 1.0f };
 GLfloat radius=9.0f;

 glPushMatrix();
    glRotatef(90.0f, 1.0f, 0.0f, 0.0f);
    glRotatef(rotateangle, 0.0f, 0.0f, 1.0f);
    glColor3f(1.0f, 0.5f, 0.3f);
    gluDisk(pObj,radius-1.0f, radius+1.0f, 20, 5);
    glPushMatrix();
       glTranslatef(0.0f, 0.0f, -0.1f);
       glRotatef(180.0f, 1.0f, 0.0f, 0.0f);
       gluDisk(pObj,radius-1.0f, radius+1.0f, 20, 5);
    glPopMatrix();

    glTranslatef(radius, 0.0f, 0.0f);
    glDisable(GL_COLOR_MATERIAL);
    if (lighton[0])
       glMaterialfv(GL_FRONT,GL_EMISSION,myred);
    else
       glMaterialfv(GL_FRONT,GL_EMISSION,myblack);
    glColor3f(1.0f, 0.0f, 0.0f);
    glMaterialfv(GL_FRONT,GL_DIFFUSE,myred);

    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
    glLightfv(GL_LIGHT1, GL_POSITION, position);
    glLightfv(GL_LIGHT1, GL_SPOT_DIRECTION, direction);

    GLfloat dx=radius*0.5;   // = radius*cos(M_PI/3);
    GLfloat dy=radius*0.8660;// = radius*sin(M_PI/3);

    glTranslatef(-radius-dx, dy, 0.0f);
    if (lighton[1])
        glMaterialfv(GL_FRONT,GL_EMISSION,mygreen);
    else
        glMaterialfv(GL_FRONT,GL_EMISSION,myblack);
    glColor3f(0.0f, 1.0f, 0.0f);
    glMaterialfv(GL_FRONT,GL_DIFFUSE,mygreen);

    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
    glLightfv(GL_LIGHT2, GL_POSITION, position);
    glLightfv(GL_LIGHT2, GL_SPOT_DIRECTION, direction);

    glTranslatef(0.0f, -2*dy, 0.0f);
    if (lighton[2])
        glMaterialfv(GL_FRONT,GL_EMISSION,myblue);
    else
        glMaterialfv(GL_FRONT,GL_EMISSION,myblack);
    glColor3f(0.0f, 0.0f, 1.0f);
    glMaterialfv(GL_FRONT,GL_DIFFUSE,myblue);
```

```
    gluCylinder(pObj,1.0f, 2.0f, 3.0f, 30, 5);
    glLightfv(GL_LIGHT3, GL_POSITION, position);
    glLightfv(GL_LIGHT3, GL_SPOT_DIRECTION, direction);

    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT,GL_EMISSION);
    glColor3f(0.0f, 0.0f, 0.0f);
    glColorMaterial(GL_FRONT,GL_DIFFUSE);

 glPopMatrix();
 if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
}
```

Similarly, make the two swing lights to have emmisive color when turn on.

Your final output should look like the below:



* END OF LAB *